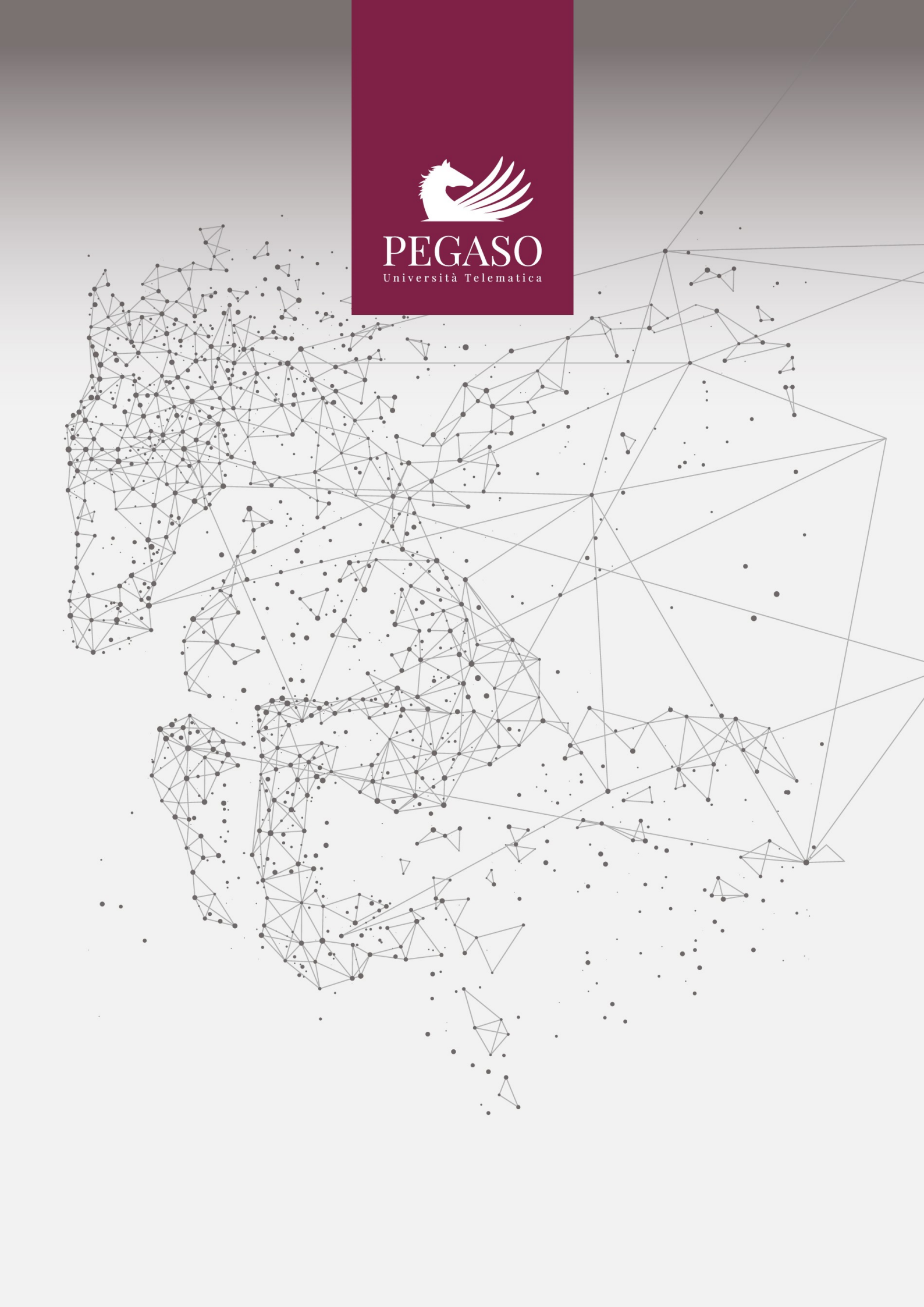

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. VISIONE AD OGGETTI DEL SOFTWARE 3

2. DEFINIRE LE CLASSI IN FASE DI ANALISI 7

3. DEFINIRE LE CLASSI IN FASE DI PROGETTAZIONE 11

BIBLIOGRAFIA 14

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Visione ad oggetti del software

Un attributo è una proprietà statica di un oggetto e per questo specificato a livello di definizione della classe:

• nome, età, peso sono attributi della classe Persona

• colore, peso, anno, modello sono attributi della classe Auto

forma, peso, colore e tipo di materiale: attributi della classe Soprammobile

I nomi degli attributi devono essere unici all'interno di una classe per evitare ambiguità. Ma se il nome è coerente con il significato semantico della proprietà non può essere altrimenti.

Ogni attributo ha un nome, un valore corrente, un insieme di valori ammessi (dominio). Ad esempio, l'attributo colore della classe Automobile può ha un dominio dei valori {bianco, nero, argento, grigio, blu, rosso, giallo, verde}. Il dominio per l'attributo peso è semplicemente quello dei numeri interi positivi. In situazioni più complesse, il dominio può essere dato a sua volta da istanze di un'altra classe. Ad esempio, sempre la classe Automobile può avere un attributo denominato motorizzazione istanza di una classe Motore, caratterizzato a sua volta da potenza, cavalliFiscali, numeroCilindri e tipoCarburante.

L'insieme degli attributi è comune a tutti gli oggetti di una stessa classe in quanto condividono la stessa semantica essendo realizzazioni concrete di uno stesso concetto, anche se in generale i valori degli attributi saranno in genere differenti nelle diverse istanze; possono anche coincidere ma è un puro caso. Ad esempio, l'attributo età ha il valore "24" nell'oggetto Franco Lorusso della classe Persona ed ha valore "32" nell'oggetto Mario Rossi sempre della classe Persona

L'insieme dei valori degli attributi ad un dato istante di tempo costituisce lo **stato** dell'oggetto. E' possibile assegnare ad un attributo un valore di default assegnato al momento dell'istanziazione dell'oggetto.

L'identità è un'altra proprietà degli oggetti di una classe che si differenzia dagli attributi per il carattere di unicità che presenta.

Nel mondo reale ogni oggetto è unico. Quando si parla di una particolare sedia si sa che si sta parlando della stessa cosa, anche se con passare del tempo alcune sua caratteristiche vengono modificate, ad esempio il costo. Eppure sempre della stessa sedia si parla. In altre parole, la sedia ha un'identità che si conserva nel tempo indipendentemente dalle modifiche che l'oggetto subisce. Viceversa, in un dato momento, oggetti diversi possono avere gli stessi valori per gli attributi ma restano comunque oggetti ben distinti, ad esempio le sedie che si usano ad uno stesso tavolo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Tutti gli oggetti quindi hanno un'identità e sono distinguibili gli uni dagli altri per la loro esistenza intrinseca e non per le proprietà descrittive che possono avere. Analogamente, gli oggetti del mondo object-oriented hanno un'identità immutabile nel tempo che li distingue e li rende unici indipendentemente dal valore dei suoi attributi che possono anche essere di uguale valore in un determinate momento per oggetti diversi.

Dal punto di vista dell'implementazione, per gestire e disporre dell'identità di un oggetto non è consigliabile definire un attributo id specifico per passarlo come parametro in una chiamata di un'operazione su quell'oggetto. Invece, l'identità è preferibilmente gestita come se fosse un attributo predefinito, unico e immutabile per tutta la vita dell'oggetto realizzato generalmente mediante un codice numerico assegnato direttamente dal sistema al momento della creazione dell'oggetto. Tale codice numerico non può essere letto ma che viene usato per tradurre ogni riferimento all'oggetto. Trattasi di una rappresentazione compatta e usata per far riferimento all'oggetto a basso livello. Definire un attributo id non è più efficiente in quanto non vi è alcun risparmio di memoria, anzi lo spazio usato può essere anche in più avendo definite e fatto uso di un attributo non necessario.

Non bisogna confondere gli identificatori interni con gli attributi del mondo reale. Gli identificatori interni sono definiti per comodità di implementazione e supportati dalla maggior parte dei linguaggi orientati agli oggetti per far riferimento agli oggetti. Ma gli identificatori interni non hanno significato nel dominio del problema. Nel paradigma orientato agli oggetti l'unità elementare di scomposizione non è più la procedura, ma l'oggetto inteso come astrazione di dati ed operazioni incapsulati in un'unica entità con un comportamento e un significato semantico strumentare a risolvere il problema che si sta trattando. Gli oggetti rappresentano entità che possono svolgere operazioni, riportare e cambiare il loro stato e comunicare con gli altri oggetti del sistema.

In un programma di tipo procedurale, si è soliti iniziare a ragionare in maniera top-down, partendo cioè dal main e creando mano a mano tutte le procedure necessarie. Tutto questo non ha più senso nella programmazione ad oggetti dove è richiesto un diverso approccio di ragionamento.

In una visione ad oggetti del software, il software deve essere pensato con una visione diversa. Dobbiamo, cioè, essere in grado di identificare gli oggetti che entrano in gioco nel programma che vogliamo sviluppare e individuare e orchestrare le interazioni fra di essi per raggiungere lo scopo del programma.

L'approccio orientato agli oggetti deve essere visto come un nuovo modo di ragionare sul software. Questa è una prospettiva diversa di quella adottata per molti anni quando con il termine object oriented (orientato agli oggetti) si è indicato più un approccio alla programmazione. In effetti l'approccio object oriented è nato con lo sviluppo di sofware che faceva uso di linguaggi di programmazione object-oriented (Smalltalk, C++, Java).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Tuttavia, ci si è anche resi conto che l'approccio l'object oriented riflette anche e soprattutto un modo di pensare alla soluzione software a 360° più che una specifica tecnica di programmazione.

Come nuovo modo di pensare la soluzione software è opportuno che sia adottato in tutte le fasi di produzione del software a partire già dalla fase di analisi dei requisiti e la progettazione dell'architettura sofware in modo che esprima tutta la sua potenza e utilità in termini di riuso del codice e manutenzione e gestione di progetti di grande dimensioni.

Si fa notare che la parte più gravosa dello sviluppo del software è la manipolazione dell'essenza del problema a causa della sua difficoltà intrinseca, ovvero l'indentificazione, l'esplorazione e l'organizzazione dei concetti del dominio dell'applicazione, piuttosto che il lavoro di codifica in un linguaggio di programmazione. Quindi la fase di l'identificazione delle classi è molto cruciale, così come capire che comportamento attribuire loro (astrazione, operazioni, interfaccia, ruolo nel contesto del programma) che poi si riflette il modo con cui poi si fanno interagire gli oggetti.

L'approccio orientato agli oggetti per la produzione del software non prende decisioni dettagliate di progetto e di implementazione prima di aver compreso bene il problema e punta ad uno stile di progettazione indipendente dal linguaggio che demanda i dettagli dell'implementazione allo stadio di codifica che è comunque relativamente meccanico.

Come nuovo modo di ragionare sul software il paradigma orientato agli oggetti deve riguardare l'intero processo di ingegneria del software per avere un impatto significativamente positivo sul processo di produzione del software e in ultima analisi sulla qualità del software.

L'approccio orientato agli oggetti usato nell'analisi, progettazione e sviluppo software è utile, in fase di analisi, per la comprensione dei problemi, e nella comunicazione con gli utenti o committenti del software. In questa fase, l'uso dell'astrazione porta chiarezza concettuale e alla formalizzazione di un vocabolario condividso riguardo concetti del dominio dell'applicazione fra i progettisti e gli utenti o committenti che evita di incorrere successivamente in problemi strutturali del sistema software che comporterebbero un alto impatto in termini di costi (diretti, ulteriori effort o indiretti, perdita di credibilità con il cliente). In fase di progettazione, è utile per la descrizione di modelli di sull'architettura software generale (progettazione di alto livello) e di modelli sulla soluzione software (progettazione software) con tutti i dettagli necessari all'implementazione (metodi, interfacce) e al lavoro di codifica.

La visione del software object oriented richiede un approccio evolutivo all'ingegneria del software. Infatti, sarebbe troppo difficile definire tutte i tipi di oggetti necessari per un sistema o un prodotto software in una singola interazione. Man mano che i modelli ad oggetti del'analisi e della progettazione evolvono, la necessità per nuove classi diventa chiaro. E' per questa ragione che l'aproccio interativo appena descritto è quello che più si addice all'approccio orientato agli oggetti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Da quanto detto, gli oggetti (e le relative classi) sono individuati e raffinati attraverso le varie fasi di produzione del software a partire dalla fase di analisi e progettazione (dove si opera all'interno del dominio dell'applicazione) fino all'implementazione del sistema sofware (dove si opera all'interno del dominio della soluzione)[1]. Questo porta ad avere due diverse espressioni di oggetto a seconda della fase e quindi del dominio in cui si opera.

Nella fase di analisi e progettazione di alto livello si opera all'interno del dominio dell'applicazione. In questo caso per oggetto si intende l'espressione di un concetto concreto o astratto e che abbia un significato semantico strumentale rispetto al problema che stiamo trattando. E' definito tramite caratteristiche generali come il comportamento e gli attributi significativi.

Nella fase di progettazione software si opera all'interno del dominio della soluzione. In questo caso per oggetto si intende un'entità del sistema software che incapsula dati e operazioni su tali dati e conseguentemente un'interfaccia per l'invocazinoe delle operazioni da altri oggetti. E' inteso soprattutto come astrazioni di un dato con le procedure associate per l'accesso dall'esterno in modo da rispondere a richieste di elaborazioni e/o di accesso ai dati (stato).

Ciò che cambia dalla fase di analisi e progettazione è il livello di astrazione (ovvero livello di dettaglio) ma non i fondamenti dell'approccio. Anche la notazione usata è la stessa, ovvero UML, ma usata da due prospettive differenti (ovvero da una prospettiva concettuale oppure software) e quindi con due diverse modalità (UML come abbozzo o UML come progetto, rispettivamente).

1 Il Dominio di Applicazione rappresenta tutti gli aspetti del problema dell'utente finale. Viene analizzato principalmente in fase di analisi dei requisiti. Viene valutato l'ambiente nel quale il sistema opera, chi sono gli utenti, quali i processi di business, dominio utente e dominio clienti.
Il Dominio di Soluzione rappresenta tutti gli aspetti del progetto esecutivo tramite il quale si risolve il problema dell'utente finale. Come tale bisogna in particolare tener conto di tutte le tecnologie disponibili per costruire il sistema. Coincide con la fase di progettazione ed esecuzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Definire le classi in fase di analisi

Gli elementi di un modello ad oggetti (classi ed oggetti, attribute, operazioni e messaggi) sono stati discussi precedentemente. Ma come facciamo ad identificare questi elementi per una specifica realtà?

Se ci si guarda attorno in una stanza, ci sono molti oggetti fisic che possono facilmente essere identificati, classificati e definiti (in termini di attributi ed operazioni). Ma quando ci si guarda attorno nello spazio del dominio di un'applicazione sofware, è molto più difficile comprendere quali siano gli oggetti rilevanti.

Il problema veramente difficile nella metodologia orientata agli oggetti è scoprire per prima cosa chi sono gli oggetti o le classi adeguate da definire. L'identificazione degli oggetti è un processo difficile che richiede una comprensione profonda del dominio applicativo.

Per scoprire gli oggetti ci si può aiutare con modelli della stessa tipologia per categorie di problema/sistema/dominio analoghi nonché basarsi sulla propria personale esperienza. Un altro buon punto di partenza è esaminare la descrizione del problema da risolvere o del dominio da modellare e farne l'analisi grammaticale.

Dalla descrizione del problema da risolvere o del dominio da modellare si effettua l'analisi grammaticale. L'idea di base è che i nomi permetteranno di individuare gli oggetti e gli attributi, mentre i verbi serviranno a farsi un'idea delle possibili operazioni e scambi di messsaggi. Gli oggetti e i gruppi di attributi ed operazioni raggruppati per oggetti suggeriranno le classi.

Vediamo un esempio a riguado.

Consideriamo l'esempio di un dominio applicativo del problema che riguarda la gestione ordine in un'azienda manufatturiera.

In Fig. 1 possiamo vedere dei frammenti dei requisiti che sono stati formulati alla fine di una serie di incontri tra il progettista e il capo dell'azienda. Da qui si può capire quanto la fase di analisi dei requisiti e la chiara formulazione di questi sia di fondamentale importanza. Può meritare spenderci un po' più di tempo pur di averli chiari e procedere fluidamente nella fase di identificazione delle classi e successiva progettazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

… dovrà essere possibile cercare un cliente ed avere mostrati i dati anagrafici del cliente trovato

… la scheda cliente dovrà mostrare tutti i dati anagrafici ed un elenco di fornitori da cui il cliente ha già acquistato

… su richiesta dell'utente dovrà essere calcolato l'importo complessivo degli ordini fatti dal cliente nell'intervallo di tempo selezionato

… per ciascun ordine dovranno essere mostrati: nome fornitore, nome cliente, linea di appartenenza dei prodotti acquistati, importo complessivo

… il report mensile dovrà contenere per ciascun cliente: la provincia di appartenenza e il totale ordinato per ciascun fornitore

Fig. 1: Frammenti di requisiti del software

Si procede all'analisi grammaticale. Come mostrato in Fig. 2, in verde sono evidenziati i nomi e in rosso i verbi. Altri nomi sono stati evidenziati in marrone e come sostantivi sono stati interpretati come potenziali attributi in quanto qualità o informazioni da associare a qualche oggetti. In generale può essere necessario fare una selezione ignorando locuzioni che non hanno significato (es. di appartenenza, mano a mano) o raggruppando casi di omonimia.

*… dovrà essere possibile **cercare** un **cliente** ed avere **mostrati** i dati anagrafici del **cliente** trovato*

*… la **scheda cliente** dovrà **mostrare** tutti i **dati anagrafici** ed un elenco di **fornitori** da cui il **cliente** ha già acquistato*

*… su richiesta dell'utente dovrà essere **calcolato** **l'importo complessivo** degli **ordini** fatti dal **cliente** nell'intervallo di tempo **selezionato***

*… per ciascun **ordine** dovranno essere **mostrati**: nome **fornitore**, nome **cliente**, **linea di appartenenza** dei **prodotti** acquistati, **importo complessivo***

*… il **report mensile** dovrà contenere per ciascun **cliente**: la **provincia** di appartenenza e il **totale ordinato** per ciascun **fornitore***

Fig. 2: Frammenti di requisiti del software con parole chiave evidenziate ai fini dell'identificazione delle classi

Dopo un processo di analisi del testo si giunge a questa lista di massima:.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Classi (in verde)
    - Cliente, Fornitore, Ordine, Prodotto, ReportMensile, SchedaCliente
- • Attributi (in marrone)
    - Dati anagrafici cliente, nome cliente, provincia cliente
    - Nome fornitore
    - Importo ordine
    - Linea appartenenza prodotti
- • Metodi (in arancio)
    - Cercare un cliente
    - Mostrare dati anagrafici e fornitori per un cliente
    - Calcolare totale ordini per un cliente
    - Selezionare ordini in un intervallo temporale
    - Calcolare totale ordini per un cliente per ciascun fornitore per mese

Occorre per una prima bozza di classi raggruppare gli attributi ed operazioni per le classi individuate e poi capire se si rispettano tutti i requisiti oppure se altre classi sono necessarie. Infatti ci potrebbero essere delle classi nascoste oppure classi che si rendono necessarie per rispettare pienamente i requisiti. Sicuramente quelle individuate non sono tutte quelle necessarie. Infatti si fa implicitamente riferimento alla necessità di un'anagrafica sia del cliente che del fornitore e anche alla necessità di collegare clienti ai relativi fornitori. Ad esempio, potrebbe essere Azienda come classe padre di Cliente e Fornitore (anche classe astratta) per raccogliere dati come il nome, partita Iva, e provincia della sede. Anche Rubrica potrebbe essere utile per raccogliere le schede clienti ed effettuare ricerche tipo cercaCliente() come richiesto. Potrebbe anche essere necessario una classe ElencoOrdini che raggruppi gli ordini e sia di supporto per la generazione dei report come da classe corrispondente o per avere il metodo calcolare totale ordini. Per tracciare lo stato dell'ordine potrebbe essere utile una classe Stato associata alla classe Ordine.

Potrebbe anche essere utile avere un'idea della categorie di classi e ruoli che ragionevolmente dovrebbero comparire. Una lista di linee guida per il modello dell'analisi è la seguente

- Entità esterne che producono o consumano delle informazioni del sistema (altri sistemi, dispositivi, persone). Nell'esempio considerato potrebbe essere il sistema dei pagamenti che necessita di un totale ordini per ciascun cliente per pagare i vari fornitori.
- Cose che sono parte del dominio delle informazioni del sistema (rapporti, informazioni, segnali). Nell'esempio considerato il report mensile richiesto di tutti gli ordini.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Occorrenze o eventi che si verificano nel contesto di un'operazione del sistema (es. passaggio di stato, completamento di una serie di operazioni, arrivo di un messaggio). Nell'esempio considerato arrivo della richiesta riguardo all'importo complessivo dell'ordine dal cliente.
- Ruoli ricoperti da persone che interagiscono con il sistema (manager, venditore). In questo caso ci sono i cliente e fornitori.
- Unità organizzativa che sono rilevanti per il sistema (divisione, gruppo, team).
- Luoghi che definiscono il contesto e le funzioni complessive del sistema (piano di lavorazione, magazzino, piattaforma di carico)
- Strutture che possono interagire con il sistema (sensori, computer, veicolo).

Pur applicando queste (e altre) tecniche, la decisione finale per l'inclusione degli oggetti potenziali in un modello è pur sempre soggettiva e basata largamente sull'esperienza personale dell'ingegnere del software. Bisogna anche ricordare che si tratta di un'attività iterativa e che quindi valutazioni successive possono portare un oggetto che era iniziamente stato incluso ad essere eliminato e viceversa uno inizialmente scartato verrà reintrodotto. Gli oggetti rifiutati possono diventare attribute per gli oggetti scartati.

Inoltre differenti assunzioni sul problema possono portare a prendere diverse decisioni sugli oggetti (in versioni semplificate diversi oggetti potrebbero non essere necessari, ad esempio software per la gestione dei condomini versus software per la gestione particolare di un singolo condominio oppure software con funzionalità ridotte).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Definire le classi in fase di progettazione

In fase di progettazione le classi con le loro relazioni si fanno più precise e specifiche del dominio.E' fondamentale che la gerarchia delle classi corrisponda effettivamente ad una gerarchia di concetti man mano sempre più dettagliati (passando dalla fase di analisi alla fase di progettazione) cercando sempre di mantenere una coerenza semantica rispetto al problema da risolvere. In particolare è bene far buon uso della ereditarietà per specializzare i concetti e giungere quindi ad un insieme di classi che sono realizzazioni concrete di concetti del dominio della soluzione.

Infatti, come mostrato in Fig. 3, si può essere tentati, sfruttando l'ereditarietà come strumento meramente tecnico di riusare il codice in assenza di una vera specializzazione (da un punto di vista semantico) ma solo per usufruire di qualche attributo già definito. Non è detto che una classe in quanto le corrisponderebbe una stessa coppia di attributi di una altra classe possa essere specializzata da quella.

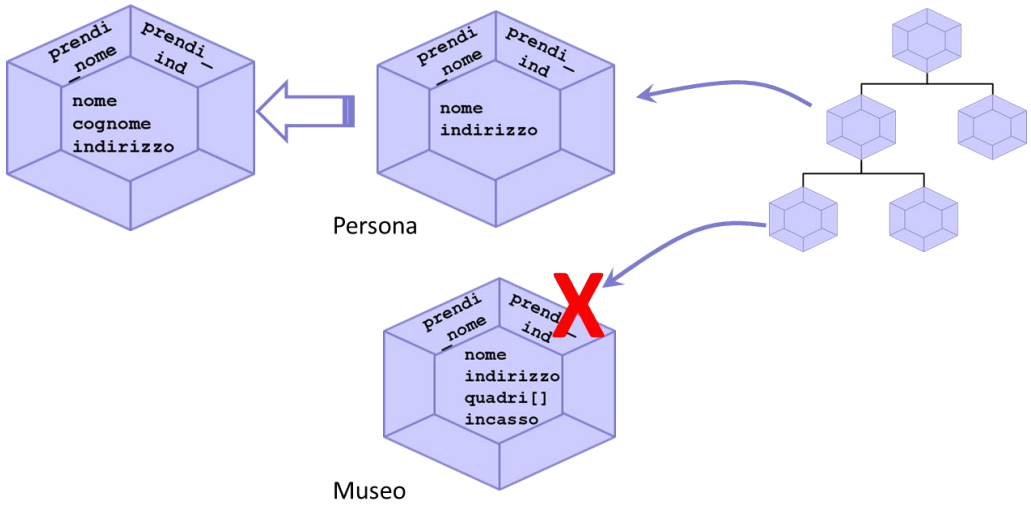


Fig. 3: Specializzazione di una classe rispettando la semantica delle classi nella gerarchia

Ad esempio, come mostrato in Fig. 4, si supponga di aver definito una classe Persona che in una fase iniziale della modellazione ha solo gli attributi nome ed indirizzo ed operazioni per leggere e scrivere tali attributi. Dovendo definire una classe Museo, anch'essa con attributi nome e indirizzo oltre a svariati altri, si potrebbe avere la tentazione di dichiarare Museo come sottoclasse di Persona in modo da risparmiarsi la fatica di ridefinire gli attributi e le operazioni comuni. In questo modo si corre il rischio che le successive modifiche alle superclassi, determinate ad esempio da una migliore comprensione del concetto rappresentato, non siano compatibili con le sottoclassi dichiarate tali solo per riusare il codice che in un certo momento era in comune ai due concetti. Se la classe introdotta non è una reale specializzazione di

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

quella nel mondo reale si correrebbe il rischio di confusione e inconsistenze. Per esempio è molto probabile che nei successivi raffinamenti la classe Persona evolva in modo incompatibile con la classe Museo (riferendosi nel mondo reale ad un concetto differente). Ad esempio una modifica molto probabile potrebbe essere sostituire l'attributo nome con nome e cognome. Attributi che un museo non ha proprio ragione di possedere.

In generale, ogni volta che una nuova classe deve essere creata ci sono diverse opzioni:

1. La classe deve essere progettata e costruita senza usare inheritance, nel caso in cui nella gerarchia non ci sia nessuna classe di cui quella in esame è una specializzazione
2. Se esiste una classe nella gerarchia di cui quella in eseme è una specializzazione, allora la nuova classe la definiamo come tale ed eredita dalla classe di livello più alto gli attribute e le operazioni e resta solo da completarla con gli attribute e le operazioni mancanti.
3. Se non esiste una classe nella gerarchia di cui quella in eseme è una specializzazione o si crea da capo (vedi sopra) oppure si può pensare in ristrutturare la gerarchia in modo che vi compaia una classe da specializzare. Non si devono compiere forzature concettuali, ma se ci sono gli estremi rendere la gerarchia più generale, allargando i concetti coperti dalla gerarchia.
4. Se esiste una classe nella gerarchia di cui quella in esame può esserne una specializzazione ma in cui una o più operazioni (o attributi) sono definite in modo diverso da come sarebbe opportuno fare nella classe da introdurre.

Altre lineeguida per la buona progettazione di oggetti e classi riguardano l'attenzione alla definizione di attribute e operazioni.

Le operazioni sono legate al ruolo e alle alle azioni previste per l'oggetto (ovvero alle responsabilità assegnate).

Le operazione devono essere in numero congruo, il che corrisponde ad avere un insieme gestibile di responsabilità per l'oggetto.

Un oggetto che abbia uno o due soli metodi deve fare riflettere. In generale bisogna evitare anche la prolifezione di micro-oggetti che possono creare confusione nel diagramma. Potrebbe essere perfettamente lecito definire un oggetto del genere (ed è sicuramente possibile farlo praticamente) ma, spesso, un oggetto creato con questi requisiti indica la necessità di "mescolarlo" con un altro oggetto con simile definizione. A maggior ragione evitare oggetti con nessun metodo. In linea di massima, un oggetto senza metodi può facilmente essere convertito in uno o più attributi da assegnare ad un altro oggetto.

Da evitare sono anche gli oggetti con troppi metodi. Assegnare ad un oggetto troppe azioni, potrebbe rendere ardua la manutenzione futura dello stesso oggetto. È consigliabile, in questo caso, cercare di spezzare l'oggetto in due oggetti più piccoli e semplificati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Le proprietà includono l'insieme dei dati dell'oggetto (attributi), ovvero le informazioni su cui i metodi possono eseguire le loro elaborazioni.

Uno degli errori più comuni che si commette quando si definiscono gli attributi di un oggetto è quello di associare ad esso quante più proprietà possibili, ritenendo che questo possa, in qualche modo, facilitare la stesura del programma. Un oggetto, invece, per essere ben definito deve contenere le proprietà che, effettivamente, gli competono e non tutte quelle che gli si potrebbero comunque attribuire.

Questa regola si rifà all'attenzione sul buon uso dell'astrazione nel modellare gli oggetti in un progetto software. Non c'è bisogno di riportare tutti i dettagli ma solo quelli che sono essenziali nella rappresentazione dell'entità nel dominio considerato in relazione al problema da risolvere. Se l'applicazione è sulla vendita di un auto allora un oggetto auto dovrà avere gli attributi su prezzo, durata della garanzia, colore, anno immatricolazione e non dati tecnici su olio da usare per il motore che interessa più nel caso si stia scrivendo un'applicazione di gestione delle riparazioni di un'autofficina. Sarebbero dettagli inutili. Per evitare, dunque, l'inconveniente di ritrovarsi dei super-oggetti sarà bene porsi la seguente domanda nella fase di definizione: «quali proprietà sono necessarie affinché l'oggetto sia in grado di eseguire le azioni identificate ai fini della risoluzione del problema?" Questa regola di buona programmazione nasce, oltretutto, dall'esigenza di rendere più facile la fase di progetto e quella di debug che altrimenti risulterebbero certamente complesse.

In questo caso ci si è riferiti alle proprietà come il set di attributi definiti per un oggetto e quindi usati come sinonimi. In realtà il termine proprietà è più generale e include in totale tre diverse accezioni:

- **attributi** rappresentano quelle proprietà che descrivono le caratteristiche peculiari di un oggetto (ad esempio, riferendoci ad una persona: altezza, peso) e modellati con i dati.
- **componenti**, invece, sono identificabili in quelle proprietà che sono atte a svolgere delle azioni (testa, corpo, mani, gambe). Sono rappresentati tramite relazione di composizione nei diagrammi UML.
- **peer objects** definiscono delle proprietà che a loro volta sono identificate e definite in altri oggetti (ad esempio: l'automobile posseduta da una persona). Sono rappresentati tramite relazioni di associazione nei diagrammi UML.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- R. S. Pressman, Principi di Ingegneria del software, McGraw-Hill, Quinta edizione

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).